Type System & Semantics

Cast

- `static_cast` — Compiler-checked, zero cost → CRTP, enum conversion
- `dynamic_cast` — Runtime-checked, vtable lookup → Not used (avoid in embedded)
- `reinterpret_cast` — Unchecked, zero cost → `Reg<Addr>` int → pointer
- `const_cast` — Unchecked, zero cost → can cast away const but not used
- `int` will cast to `uint` when added together

lvalue vs rvalue

- `lvalue` features persistent address in mem so can be pointed to
- `rvalue` temporary lives in `CPU register` during evaluation and dies at the end of expression `;`
- & test : & get the address outof `lvalue` while `rvalue` cannot

```
int x = 5;              // x is an lvalue — named, has address, persists
                        // 5 is an rvalue — temporary, no lasting identity

process(x);             // binds to int& (lvalue ref)
process(5);             // binds to int&& (rvalue ref)
process(std::move(x));  // std::move CASTS x to rvalue ref — does NOTHING at runtime
```

Syntax	Name	Binds to
`T&`	lvalue reference	alias to variable
`const T&`	const lvalue reference	read-only alias (accepts rvalue too)
`T&&`	rvalue reference	binds to temporaries

`std::move` — cast, not copy

- `std::move(x)` is equivalent to `static_cast<T&&>(x)` — a cast, not an operation
- The move constructor then *steals* heap pointers from the rvalue
- After `std::move(x)`, `x` is still a valid object in a "moved-from" state (can be destroyed or reassigned, but content is unspecified)
- `std::move(x)` produces a `T&&` (rvalue reference to x). It is literally `static_cast<T&&>(x)`.

Trivial vs non-trivial types

- **Trivial** — no constructor, no destructor, no virtual methods. Raw bytes. `memcpy` is safe.
  - `int`, `double`, `float`, `char`, `int[N]`, `struct { double x, y; }`
  - `int x = 5;` → 4 bytes on stack.
  - `int* p = new int(5);` → 8-byte pointer on stack + 4-byte int on heap.
  - The `new` version requires manual `delete` or a smart pointer to avoid leaking.
- **Non-trivial (resource-managing)** — ctor allocates heap, dtor frees it. `memcpy` → double free.
  - `std::string`, `std::vector<T>`, `std::unique_ptr<T>`

    - `memcpy` blindly copies stack bytes (ptr, size, cap) → both src and dest point to same heap addr → both destructors call `free()` → double free
- **Virtual types** — `memcpy` → wrong vptr → virtual calls jump to garbage.
    - A class with `virtual` has a hidden `vptr` pointing to its vtable. `memcpy` copies raw bytes (including vptr) without invoking the constructor that correctly sets vptr for the destination's actual type.
    - Ctor sequence for vtable: base ctor runs first and sets `vptr → Base::vtable`. Then derived ctor runs and overwrites `vptr → Derived::vtable`. This ensures virtual calls during base ctor dispatch to base methods (not yet-constructed derived), and after full construction, vptr points to the final derived vtable.

```
#include <type_traits>
static_assert(std::is_trivially_copyable_v<double>);        // ✓
static_assert(std::is_trivially_copyable_v<int[4]>);        // ✓
static_assert(!std::is_trivially_copyable_v<std::string>);  // ✓ not trivial
```

Keyword reference

Keyword	Affects	Purpose
`const` (on variable)	compiler	prevents modification, enables placing in `.rodata`
`const` (after method)	compiler	method can be called on `const&` objects; use `mutable` for exceptions
`volatile`	compiler	prevents read/write elimination — every access emits a real load/store
`inline`	linker symbol	changes symbol from strong (`T`) to weak (`W`) — allows multiple definitions
`constexpr`	compiler	enables compile-time evaluation; implies `inline`
`static` (in function)	compiler/linker	variable persists across calls; stored in `.bss`/`.data`, not stack
`static` (at file scope)	linker	internal linkage — symbol visible only in this TU (`t` in nm)
`static` (class member)	linker	one instance shared across all objects; not tied to `this`
`mutable`	compiler	allows modification of this field even in `const` methods (e.g., caches, mutexes)
`atomic`	compiler/hw	thread-safe read/write + memory ordering; single CPU instruction `dmb ish` vs complex `std::mutex`
global variable	linker	external linkage by default — visible across all TUs; stored in `.data`/`.bss`; use `extern` to declare without defining

Keyword	Affects	Purpose
`#define MACRO`	preprocessor	textual substitution in `.ii` — no symbol, no type safety

- `atomic` is lock_free while mutex is not; `dmb ish` drains all pending writes before continue
- `atomic<BigStruct>` falls back to mutex for speed

function pointer

```
__BEGIN_DECLS
void(*signal(int, void (*)(int)))(int);
__END_DECLS
#endif  /* !_SYS_SIGNAL_H_ */
```

- void(*)(int) is a function pointer

Pointer vs reference vs value

```
int x = 5;           // value on STACK — owns data directly
int* p = &x;         // pointer on STACK — holds address of x (8 bytes on 64-bit)
int& r = x;          // reference — alias, not a separate object, no extra memory

int* h = new int(10); // pointer on STACK (8 bytes) → data on HEAP (4 bytes)
```

Key distinction: `sizeof(std::vector<int>)` gives the stack footprint (~24 bytes: ptr+size+cap), NOT the heap data size. Use `.size()` for the logical element count.

Polymorphism

`class Stm32Gpio : public IGpio {`

- public : `IGpio& ref = stm32_obj;` → private does not allow this

struct, class, and namespace

- struct is public by default; class is private by default
- struct defaults to public inheritance; class defaults to private inheritance
- Convention: `struct` for passive data (all public fields, no invariants)
- Convention: `class` when there are invariants, private state, or methods that enforce rules

namespace vs struct

- `namespace` can be extended anywhere
- `namespace` cannot have `template` parameters, unlike the locally closed struct

```
// valued template !
template<uint32_t Addr>
```

```
struct Reg {
    // & makes ref() a lvalue alias to the memory at Addr
    static volatile uint32_t& ref() {
        return *reinterpret_cast<volatile uint32_t*>(Addr);
    }
    static void set_bits(uint32_t mask) { ref() |= mask; }
    static void clear_bits(uint32_t mask) { ref() &= ~mask; }
};

// constexpr guarantees compile-time fixed, thus can be templated
constexpr uint32_t GPIOA_ODR = 0x40020014;
Reg<GPIOA_ODR>::set_bits(1 << 5);  // set pin 5 high
```

Virtual, vtable, and override

- `virtual` prefixed base method should be overridden by derived method
- `virtual uint64_t micros() = 0;` → `=0` means pure virtual where base cannot provide implementation
- `override`-suffixed derived method lets compiler safety check against wrong paras
- Correct ctor sequence: `new Derived` → allocate heap → run base ctor (sets vptr to Base's vtable) → run derived ctor (overwrites vptr to Derived's vtable). Final vptr points to Derived's vtable.
- With `new`: object on heap, accessed via base pointer, virtual dispatch (indirect call via vtable). Without `new`: `Stm32Gpio gpio(reg, 5);` lives on stack, compiler knows the exact type, can devirtualize → direct call, no vtable overhead.

vtable layout and dispatch cost

```
IGpio* gpio = new Stm32Gpio(reg, 5);
gpio->set(true);   // which set() gets called?

// .rodata (compiled into binary, lives forever):
// ┌──────────────────────────────────────────────┐
// │  vtable for Stm32Gpio:                       │
// │    [0] → Stm32Gpio::~Stm32Gpio()  (.text)    │
// │    [1] → Stm32Gpio::set()         (.text)    │
// │    [2] → Stm32Gpio::read()        (.text)    │
// └──────────────────────────────────────────────┘

// Heap:
// ┌──────────────────────────┐
// │  vptr ──→ vtable above   │  8 bytes (hidden first field)
// │  reg_ = 0x40020014       │  8 bytes
// │  pin_ = 5                │  1 byte + 7 padding
// └──────────────────────────┘
```

`gpio->set(true)` compiles to:

1. Load vptr from object (memory read)
2. Load vtable[1] (memory read — get Stm32Gpio::set address)
3. Call that address (indirect branch)

vs. non-virtual call:

1. Call Stm32Gpio::set() (direct branch — address known at compile time)

Cost: 2 extra memory reads + indirect branch ≈ ~5ns overhead

dependency injection → polymorphism at consumer

```
constexpr uint32_t GPIOA_ODR = 0x40020014;
Stm32Gpio led(GPIOA_ODR, 5);

// when Stm32Gpio is the only child
void blink(Stm32Gpio& led) {led.set(true); }
// vtable lookup, works with any GPIO
void blink(Gpio& led) {led.set(true); }
```

duck-typing compile time polymorphism

- no vtable → can be constexpr & no base class
- but keeps one fn copy per type and no mixed typed arr

```
template<typename Gpio>
void blink(Gpio& led) {led.set(true); }
```

CRTP compile time polymorphism

- no mixed typed arr
- GpioBase and GpioBase are different essentially
- `*this` dereferences the base pointer to get a reference. `static_cast<Derived&>(*this)` casts the base *reference* to a derived reference. Without `*`, you'd cast a pointer, which is also valid (`static_cast<Derived*>(this)`) but less idiomatic for member access.

```
template<typename Derived>
class GpioBase {
public:
    // toggle body is identical for all derived
    void toggle() {
        // cast base [this] -> derived [self]
        auto& self = static_cast<Derived&>(*this);
        self.set(!self.read());
    }
};

class Stm32Gpio : public GpioBase<Stm32Gpio> {
public:
    void set(bool high) { /* register write */ }
    bool read() { return /* register read */; }
```

```
    // toggle() inherited from GpioBase — calls Stm32Gpio::set/read directly
};
```

Stack, Heap, and Object Lifecycle

Fundamentals

reference's hidden pointer and a heap pointer live on the stack.

What is a stack frame?

Every function call pushes a **frame** onto the stack containing: - Return address after fn returns - Arguments passed to the function - Local variables declared inside the function

When the function returns, the frame is **popped** — all locals are destroyed instantly.

- **Stack** (~1 instruction each, ~100× faster)
  - Allocate: sub sp, #size
  - Free: add sp, #size
- **Heap** (function call → allocator bookkeeping)
  - Allocate: bl _malloc → free-list search, possible mmap syscall
  - Free: bl _free → coalesce free-list, return block
  - Heap overhead is primarily the free-list search + metadata bookkeeping in malloc/free, not virtual memory paging. The allocator must traverse the free-list, split/coalesce blocks, and maintain headers. Stack is just a pointer bump (sub sp).

```
call chain: main() → foo() → bar()

Stack (grows downward):
┌──────────────────┐  high address
│  main() frame    │  x, v, s, gpio
├──────────────────┤
│  foo() frame     │  local, temp
├──────────────────┤
│  bar() frame     │  i, j        ← sp (stack pointer) points here
└──────────────────┘  low address

bar() returns → sp moves up → bar's frame is gone
foo() returns → sp moves up → foo's frame is gone
```

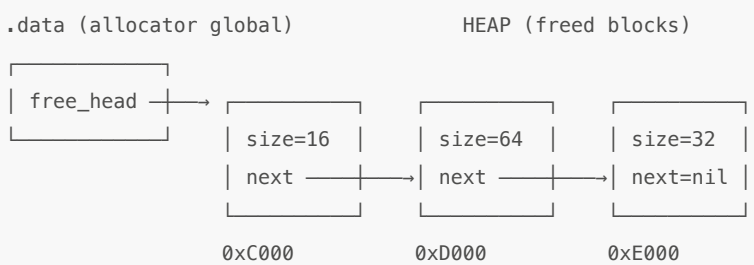
What is the heap?

Memory pool managed by malloc/free (C) or new/delete (C++). Allocate/free by user or smart containers (std::vector<T>, std::unique_ptr<T>).

The allocator (libmalloc) maintains a **free-list** — a linked list stored inside freed heap blocks:

- head pointer: lives in allocator globals (.data)

- `next` pointers: live inside each freed block (reusing the now-unused data area)



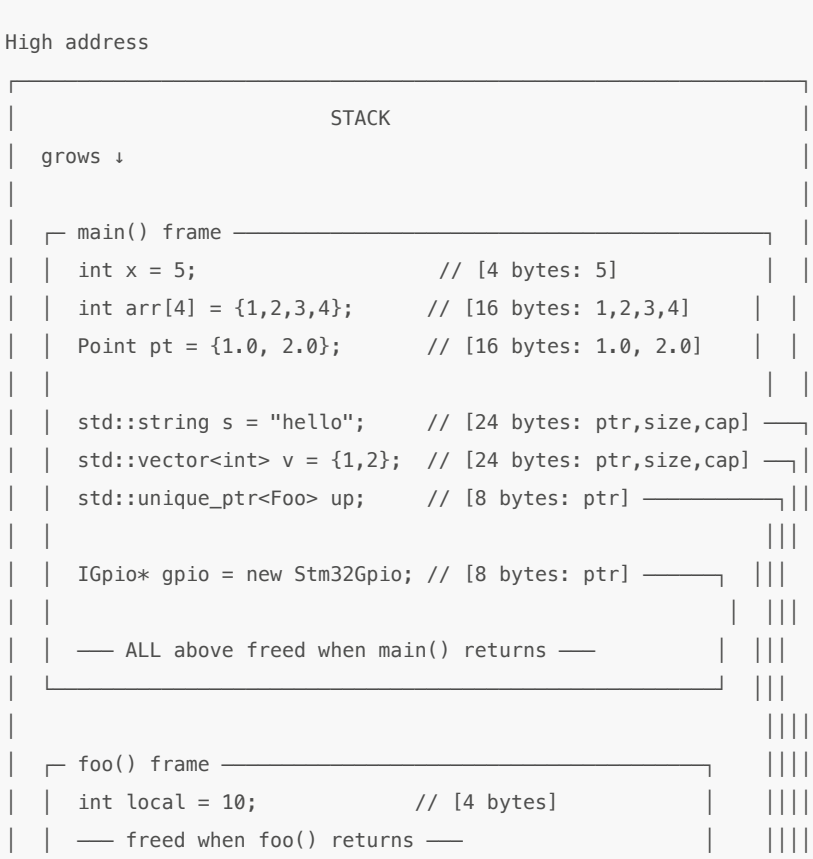
- `malloc(20)`: walk free-list → find block ≥ 20B → split → return pointer
- `free(ptr)`: mark block unused → write `next` pointer into it → re-link list

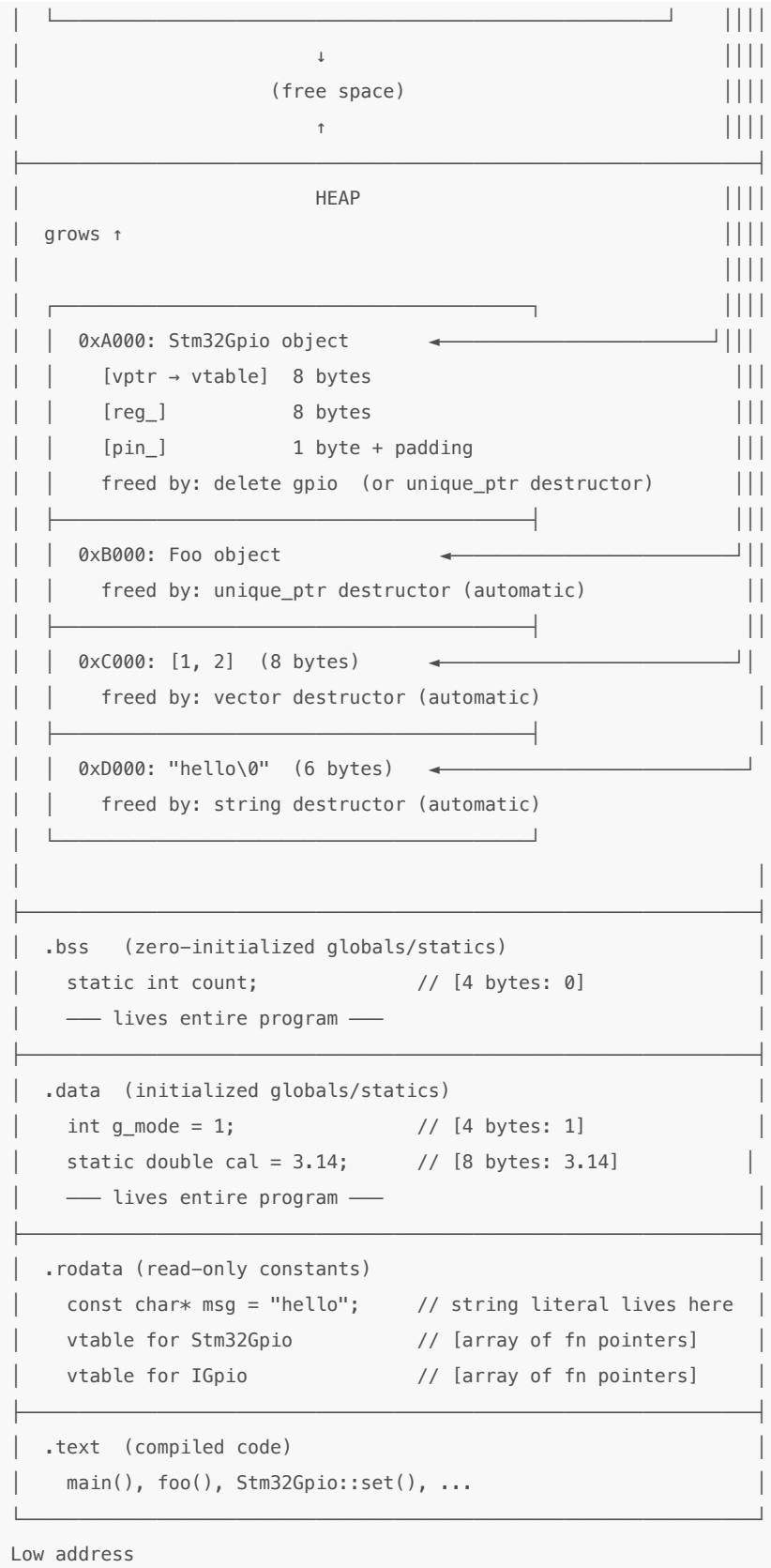
When do we need data on heap

Stack is preferred but:

1. **Lifetime must outlive the scope** — returning a large object from a factory, storing in a container that persists after the creating function returns.
2. **Size unknown at compile time** — `new int[n]` where `n` is runtime. Stack requires compile-time size (VLAs are non-standard).
3. **Too large for stack** — stack is typically 1–8 MB. A 10 MB array must go on heap.
4. **Polymorphism via base pointer** — `ISensor* s = new AdcSensor();` — the concrete type is decided at runtime. Stack allocation requires knowing the exact type.

Full memory layout — where everything lives





Rules of thumb

- Local variable → stack (automatic, fast, no leak possible)
- `global` / `static` → .data or .bss (lives forever)
    - `global` has external linkage (visible in all TUs via `extern`); `static` at file scope has internal linkage (visible only in this TU). Both live in `.data`/`.bss` for the program's lifetime.

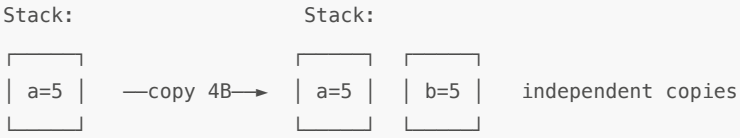
- `static` fns
- `new` / `malloc` → heap (manual or smart-pointer managed)
- String literals (`"hello"`) → .rodata (compiled in, read-only)
- vtables → .rodata (one per class, compiled in)
- Code → .text (compiled in, read-only, executable)
- If a type has a destructor → don't `memcpy`, use copy/move constructors

---

Copy vs move

Trivial types — memcpy is the entire operation

```
int a = 5;
int b = a;          // compiler emits: memcpy(&b, &a, 4)  — done
```



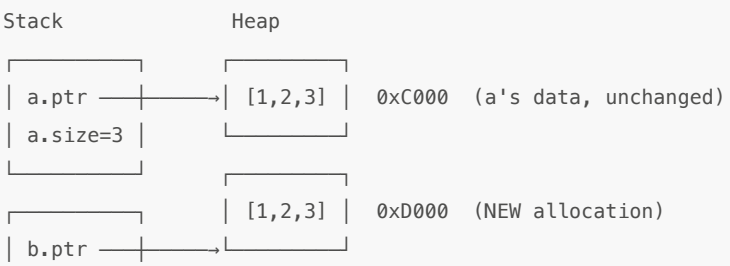
Non-trivial types — constructor must run, NOT memcpy

```
std::vector<int> a = {1, 2, 3};

// Stack              Heap
// ┌────────────┐     ┌──────────┐
// │ a.ptr ─────┼────→│ [1,2,3] │  0xC000
// │ a.size=3 │      └──────────┘
// │ a.cap=3  │
// └────────────┘
```

- `std::vector` is NOT a linked list. It is a contiguous dynamic array (single heap block). `std::list` is the doubly-linked list. Vector gives O(1) random access; list gives O(1) insert/remove at iterators.

Copy: `vector<int> b = a;`



```
| b.size=3 |
└──────────┘
```

- Copy constructor: allocate new heap block, memcpy the CONTENTS
- Cost: malloc() + memcpy(N elements)
- `malloc` returns a *new, distinct* heap address from the free-list. Now `a.ptr` and `b.ptr` point to different blocks — no double free.

Move: `vector<int> b = std::move(a);`

```
Stack              Heap

┌──────────┐
| a.ptr=nil|      ┌──────────┐
| a.size=0 |      | [1,2,3] |  0xC000  (same block, new owner)
└──────────┘      └──────────┘

┌──────────┐           ↑
| b.ptr ──┼───────────┘
| b.size=3 |
└──────────┘
```

- Move constructor: steal pointer, null source
- Cost: 3 pointer/int assignments (24 bytes)
- a is still alive and valid — just empty

C++ Build Pipeline

```
Source Code          Stage 1          Stage 2          Stage 3          Stage 4
   .cpp      ──────→ .i/ii   ──────→ .s       ──────→ .o       ──────→ executable
              PREPROCESS       COMPILE          ASSEMBLE          LINK

  main.cpp    main.ii          main.s           main.o           cvd_controller
  (text)      (expanded text)  (assembly text)  (machine code)   (runnable binary)
```

Stage 1: PREPROCESS — cpp (or g++ -E)

- #include parts of .cpp are expanded by including .h into .ii
- CMake's target_include_directories becomes the -I flag
- cpp -I src/common src/controller/main.cpp -o main.ii
- g++ -E -I src/common src/controller/main.cpp -o main.ii

Stage 2: COMPILE — g++ -S

- g++ -S -std=c++17 -I src/common src/controller/main.cpp -o main.s

Stage 3: ASSEMBLE — g++ -c (or 'as')

- g++ -c -std=c++17 -I src/common src/controller/main.cpp -o main.o

Stage 4: LINK — g++ (calls ld internally)

- The dynamic loader path is stored in the ELF `PT_INTERP` segment (e.g., `/lib64/ld-linux-x86-64.so.2` on Linux, `/usr/lib/dyld` on macOS). The kernel reads this at exec time to determine which loader runs.
- g++ main.o -o main
- file main → check the dynamic loader

All at once:

- g++ -std=c++17 -I src/common src/controller/main.cpp -o main

Compilation details

- Lexing & Parser
  - Tokenize source → build Abstract Syntax Tree (AST)
  - tree structure reveals the sequence for execution
- Semantic analysis
  - type checking, access control
  - Verify pkt.temperature is double, send() args match signature
- Template instantiation
  - replace the `template <typename T>` arg
- Overload resolution
  - std::cout << pkt.temperature → pick operator<<(double)
- Optimization (-O2)
  - Inline functions, eliminate dead code, unroll loops, vectorize
- Register allocation
  - Map variables to CPU registers (rax, xmm0, etc.)
- Instruction selection
  - Choose x86 instructions (movsd, addsd, subsd)
- Struct layout
  - Compute field offsets: temperature is at byte 12 from struct start
- Stack frame layout
  - Decide where each local variable lives relative to Register Stack Pointer

inline, constexpr, and inlining

- `inline`, `constexpr`, `template` all produce weak symbols → all ODR-safe in headers
  - each .o carries a copy → linker keeps one → discards duplicates
  - any definition placed in .h MUST be one of these, otherwise ODR violation
- `constexpr` → implies `inline` (weak symbol) + CAN evaluate at compile time
  - `constexpr int x = factorial(5);` → computed by compiler, no runtime call
  - runtime args → falls back to normal (inlined) function call
- Inlining (body expanded at call site, no `bl`) is a compiler optimization
  - `-O2` inlines small fns automatically regardless of `inline` keyword
  - too much inlining bloats code → cache misses → slower
  - `-O2` also folds obvious compile-time constants
  - `volatile` prevents reordering or cutting of loads/stores of a var

Link (ld)

- Linker resolves compile-yielded symbols by name — it does not re-check types.

1. Link time
    - Linker marks executable as dynamically linked.
    - writes metadata to executable
        - PT_INTERP (loader path, e.g. ld-linux-x86-64.so.2)
        - DT_NEEDED entries (required shared libs)
2. Runtime (after build, when you run executable)
    - Kernel reads PT_INTERP and starts the dynamic loader.
    - Loader resolves shared libraries and verifies arch / version

- ODR violation occurs when two .o files are compiled against different versions of the same header — the linker silently picks one definition, and the other .o interprets memory with wrong layout → undefined behavior.

.o / .so / .dylib / .dll comparison

- `.o` — relocatable object file (one TU's machine code, unresolved symbols, build-time only)
- `.so` — Linux shared library (linked, PIC, runtime-loadable via `dlopen`)
- `.dylib` — macOS shared library (equivalent to `.so`)
- `.dll` — Windows shared library (equivalent to `.so`)
- `ctypes.CDLL` — Python's FFI wrapper that calls `dlopen`/`LoadLibrary` to load a `.so`/`.dylib`/`.dll`
- `ldd` — Linux CLI tool that prints shared library dependencies of an executable (macOS equivalent: `otool -L`)

Property	.o (object file)	.so / .dylib / .dll (shared library)
Symbols	unresolved (U = undefined)	all resolved
Runtime loadable	no	yes (dlopen / ctypes.CDLL / LoadLibrary)
Contents	one .cpp's machine code	linked, complete library
Position-independent	not required	yes, typically needs -fPIC
Primary use	linker (ld) at build time	OS loader at runtime

.so properties:

- extern "C" for Python/C interop (no mangling)
- g++ -std=c++17 -shared -fPIC pid_lib.cpp -o libpid.dylib

Mangling and symbols

- One executable, one symbol
- Mangling distinguishes overloaded function names in C++

```
  _ZN3PID7computeEddd
   || |   |     ||||
```

```
      || |   |      |└── d = double (paras)
      || |   |      └─── E = end of nested name
      || |   └────────── 7compute = method name
      || └────────────── 3PID = class name
      |└──────────────── N = nested name start
      └───────────────── _Z = C++ mangled
```

- `extern "C" void handle_signal(int sig);` disables mangling so the symbol is `handle_signal` only
- `nm main.o`
    - `T` global function / `t` static (internal linkage) function
    - `b` static local data
    - `r` string literals and constants
    - `U` undefined — should be linked at link time
    - `W` weak symbols — inline, constexpr, template
- weak symbols
    - inline functions are intrinsically weak
    - compiler randomly keeps one of multiple conflicting weak symbols

Header files & ODR

- `<T>` must be in .h because compiler needs the body to instantiate per type
    - escape hatch: explicit instantiation in one .cc keeps template body out of header
- any non-template definition in .h must be marked `inline` to be ODR-safe
    - class member functions defined inside class body are implicitly `inline`
- `#ifndef FOO_H_` / `#define FOO_H_` / `#endif` guard prevents multiple inclusion
    - With include guards, the first `#include` seen by the preprocessor wins; subsequent includes of the same header are skipped (guarded out).
- one .h for all .cpp can avoid ODR (One Definition Rule)
    - Bazel / CMake guarantees single header version so ODR is avoided
    - If two .cpp files include different versions of the same header (e.g., struct layout changed), each .o is compiled with a different definition of the same symbol. The linker silently picks one — the other .o now interprets memory with the wrong layout → undefined behavior (silent corruption, not a linker error).

__restrict

- these pointers do not reference one another during function call
- do not bl hence compiler friendly

```
int getaddrinfo(const char * __restrict, const char * __restrict,
                const struct addrinfo * __restrict,
                struct addrinfo ** __restrict);
```

g++ invocation

```
g++ -v -std=c++17 -I src/common src/controller/main.cpp -o main 2>&1
Apple clang version 17.0.0 (clang-1700.6.3.2)
Target: arm64-apple-darwin25.4.0
Thread model: posix
InstalledDir: /Library/Developer/CommandLineTools/usr/bin
```

-std=c++17 & POSIX & libc

- std flag specifies which path stds should be found
- POSIX libs are OS specific and can be checked by `g++ -###`

	std:: (C++)	libc (C)	POSIX
Defined by	ISO C++ committee	ISO C committee	IEEE / Open Group
Name mangling	yes (`_ZSt5clamp...`)	no (`memcpy`)	no (`signal`)
Namespace	`std::`	`std::` or global	global only
Templates/overloads	yes	no	no (supports both C/C++)
Talks to kernel	rarely	sometimes (`malloc`)	almost always
Portable to Windows	yes (MSVC)	yes (MSVC)	no

Embedded C++ Patterns

1. RAII scope guard — auto-release hardware resources

```cpp
struct CriticalSection {
    CriticalSection()  { __disable_irq(); }
    ~CriticalSection() { __enable_irq(); }
};

void transfer() {
    CriticalSection cs;          // interrupts disabled
    shared_buf[idx++] = data;
}                                // ~CriticalSection() re-enables — even if early return

// Generic scope guard (C++17)
template<typename F>
struct ScopeGuard {
    F fn;
    ~ScopeGuard() { fn(); }
};
// usage: ScopeGuard guard{[] { HAL_SPI_Release(); }};
```

2. ISR ↔ main communication

```
// ISR sets flag — main polls
volatile bool data_ready = false;

// ISR (runs in interrupt context — no heap, no blocking)
extern "C" void USART1_IRQHandler() {
    rx_buf.push(USART1->DR);   // lock-free ring buffer
    data_ready = true;
}

// main loop
while (true) {
    if (data_ready) {
        data_ready = false;
        process(rx_buf.pop());
    }
    __WFI();  // sleep until next interrupt
}
```

3. Ring buffer — lock-free, fixed-size, ISR-safe

```
template<typename T, size_t N>
struct RingBuffer {
    static_assert((N & (N - 1)) == 0, "N must be power of 2");
    T buf[N]{};
    volatile size_t head = 0;  // written by producer (ISR)
    volatile size_t tail = 0;  // written by consumer (main)

    bool push(T val) {
        size_t next = (head + 1) & (N - 1);
        if (next == tail) return false;  // full
        buf[head] = val;
        head = next;
        return true;
    }
    bool pop(T& out) {
        if (head == tail) return false;  // empty
        out = buf[tail];
        tail = (tail + 1) & (N - 1);
        return true;
    }
};
```

4. Static / pool allocation — no heap

```
// Fixed pool — no malloc, no fragmentation
template<typename T, size_t N>
struct Pool {
    std::aligned_storage_t<sizeof(T), alignof(T)> storage[N];
```

```
    bool used[N]{};

    T* alloc() {
        for (size_t i = 0; i < N; ++i)
            if (!used[i]) { used[i] = true; return reinterpret_cast<T*>(&storage[i]); }
        return nullptr;
    }
    void free(T* p) {
        size_t i = reinterpret_cast<std::aligned_storage_t<sizeof(T), alignof(T)>*>(p) - storage;
        p->~T();
        used[i] = false;
    }
};

Pool<Packet, 16> pkt_pool;  // 16 Packets, all on stack/static
```

5. Placement new — construct at specific memory

```
// DMA buffer at fixed address
alignas(32) uint8_t dma_buf[sizeof(Packet)];

Packet* p = new (dma_buf) Packet{.id = 1, .temp = 36.5};  // no malloc
// ... use p ...
p->~Packet();  // explicit destructor, no delete
```

6. State machine — enum + transition function

```
enum class State { Idle, Heating, Stabilize, Processing, Fault };
enum class Event { Start, TempReached, Done, Error, Reset };

State transition(State s, Event e) {
    switch (s) {
        case State::Idle:
            if (e == Event::Start) return State::Heating;
            break;
        case State::Heating:
            if (e == Event::TempReached) return State::Stabilize;
            if (e == Event::Error) return State::Fault;
            break;
        case State::Stabilize:
            if (e == Event::Done) return State::Processing;
            break;
        case State::Fault:
            if (e == Event::Reset) return State::Idle;
            break;
        default: break;
    }
    return s;  // no transition
}
```

7. Type-safe units — prevent mixing

```
struct Milliseconds { uint32_t v; };
struct Microseconds { uint32_t v; };
struct Celsius      { double v; };

void delay(Milliseconds ms);         // can't accidentally pass µs
void set_temp(Celsius target);       // can't pass raw double

// compile error: delay(Microseconds{100});
```

8. Double buffering — DMA ping-pong

- DMA with double buffering completes the data read for CPU → saves polling

```
alignas(32) uint16_t adc_buf[2][256];  // two buffers
volatile int active = 0;               // DMA writes to active

extern "C" void DMA1_IRQHandler() {
    active ^= 1;                       // swap
    DMA1->M0AR = (uint32_t)adc_buf[active];  // point DMA to new buffer
}

// main processes the inactive buffer — no race
void process() {
```

```
    uint16_t* safe = adc_buf[active ^ 1];
    for (int i = 0; i < 256; ++i) { /* use safe[i] */ }
}
```

9. Error handling without exceptions

```
enum class Err { Ok, Timeout, CrcFail, BusError };

struct Result {
    double value;
    Err err;
    explicit operator bool() const { return err == Err::Ok; }
};

Result read_sensor(uint8_t addr) {
    if (!i2c_start(addr)) return {0, Err::BusError};
    uint16_t raw = i2c_read16();
    if (!verify_crc(raw))  return {0, Err::CrcFail};
    return {raw * 0.01, Err::Ok};
}

// usage
if (auto r = read_sensor(0x48); r) {
    temperature = r.value;
} else {
    handle_error(r.err);
}
```

10. Compile-time platform selection

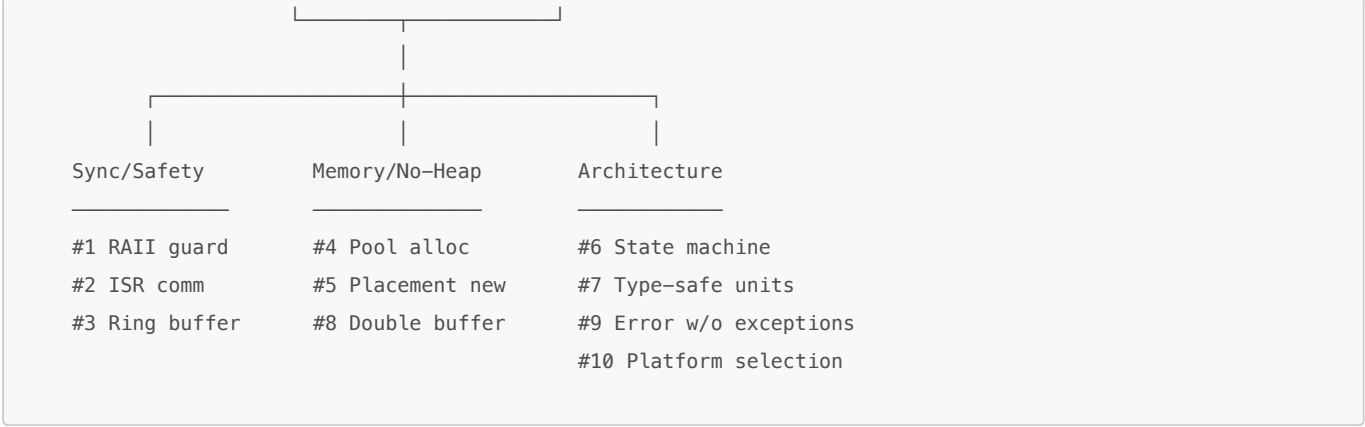
```
template<typename MCU> struct Gpio;

struct Stm32F4 {};
struct Stm32H7 {};

template<> struct Gpio<Stm32F4> {
    static void set(int pin) { GPIOA->BSRR = (1 << pin); }
};
template<> struct Gpio<Stm32H7> {
    static void set(int pin) { GPIOA->BSRRL = (1 << pin); }
};

using Led = Gpio<Stm32F4>;  // single point of platform change
Led::set(5);
```

```
                ┌──────────────────────┐
                │  ISR ↔ Main Loop     │  ← the core constraint
```


Sync/Safety
Memory/No-Heap
Architecture
#1 RAII guard
#2 ISR comm
#3 Ring buffer
#4 Pool alloc
#5 Placement new
#8 Double buffer
#6 State machine
#7 Type-safe units
#9 Error w/o exceptions
#10 Platform selection